

Relatório de Falhas e Pendências

Sistema Promudas — PDV / Gestão de Encomendas

Data do levantamento: 06/08/2026

Baseado em: leitura da memória do projeto + inspeção do código/schema/histórico git atuais.

1. Resumo executivo

O backend foi migrado para produção (servidor Ubuntu) em 25/06/2026, com importação do histórico real de clientes/pedidos/pagamentos a partir de planilha Excel. O último commit no repositório é de 26/06/2026 ("nova versão") — não há nenhum commit nem sessão de trabalho registrada nas quase seis semanas entre essa data e hoje (06/08/2026).

Essa lacuna pode ser só sinal de estabilidade (o sistema rodando bem em produção, sem necessidade de mudanças), mas também pode significar que bugs de uso não estão sendo capturados ou repassados. Vale confirmar diretamente com o dono qual dos dois cenários é o real.

O achado mais importante deste levantamento é de segurança: a rota de cadastro de usuário segue pública em produção (seção 2), uma correção que já tinha sido desenhada e aprovada, mas ficou pendente de aplicação no momento da migração ao servidor — e não foi aplicada.

2. Achado crítico — rota de cadastro pública em produção

POST /api/auth/registrar não exige token JWT. Em src/server.js, o router de auth é montado na linha 31, antes do middleware verificarToken (linha 42); e src/routes/authRoutes.js expõe a rota /registrar sem nenhum middleware de proteção.

Risco concreto

Qualquer pessoa com acesso à rede onde o backend está exposto (hoje via Cloudflare Tunnel, segundo decisão registrada em 16/06) pode criar uma conta própria e obter um token JWT com acesso total: listar/editar/excluir clientes, pedidos e pagamentos — dados financeiros e de clientes reais, já que o sistema está em produção com dados importados.

Contexto

Essa pendência foi identificada em 16/06/2026, com correção já definida e aprovada pelo dono: exigir verificarToken só na rota /registrar, mantendo /login público — mudança de uma linha em authRoutes.js. A decisão na época foi adiar a aplicação "para quando migrar para o servidor". A migração para o servidor aconteceu em 25/06/2026, mas o ajuste na rota não foi feito junto.

Recomendação

Aplicar a correção assim que possível. É uma mudança pontual e de baixo risco (adicionar verificarToken como segundo argumento da rota /registrar), mas exige redeploy manual no servidor (o backend não se autoatualiza — só o app Windows tem auto-update).

3. Pendências técnicas confirmadas no código atual

- pagamentos.data_pagamento — nullable de propósito (decisão de 23/06), com fallback manual para criado_em em relatorioService.js (linha ~32, com TODO explícito no código). Não é bug — só lembrete de que a decisão foi conscientemente mantida assim.
- pedidos.status_geral — coluna default "Ativa", nunca mais lida ou alterada depois da criação do pedido; redundante com o soft-delete via campo "ativo". Falta decidir: dar um ciclo de vida real (Ativa/Cancelada/Concluída) ou remover a coluna.
- pagamentos.recebedor — o dono já havia decidido remover essa coluna (não pretende usá-la); ainda consta no schema.
- Estoque de produtos — não implementado. A tabela produtos só tem nome, preço e ativo; não há controle de quantidade nem baixa automática ao vender/entregar.
- Gestão de usuários e permissões por módulo — usuarios segue isolado, sem campo de perfil/papel. Era a "próxima etapa" planejada após a separação PDV/Entregas/Administração (iniciada em 22-23/06), mas nunca foi começada.
- Decisão sobre rodar offline — segue em aberto (análise feita em 16/06, ver Relatorio-Estado-Projeto.pdf,

Parte III). Recomendação registrada foi resolver por rede (LAN física + ZeroTier) em vez de reescrever o app para offline-first.

- Comentário desatualizado em `carrinho_service.dart` (linha 3): "TODO: substituir por SQLite — persistir o carrinho...". Contradiz a decisão já tomada e executada de usar API direta (sqlite foi removido do pubspec na Fase 2 da refatoração). É só um comentário órfão, mas pode confundir uma leitura futura do arquivo.

4. Funcionalidades sem confirmação de teste em produção

Estas features foram implementadas e ficaram registradas como "não testadas pelo dono" no momento da implementação. Como não há relatos de bugs nem commits desde 26/06, e várias delas tiveram commits de acompanhamento em 25-26/06 que sugerem uso real (ex.: "Agora o programa aceita troca como forma de pagamento", "botando dinheiro como forma de pagamento padrão"), é provável que estejam validadas — mas nenhuma memória foi atualizada para confirmar isso.

- Cheques a depositar — fluxo completo (registrar cheque no PDV !' depositar !' conta de destino).
- Escambo (troca por pimenta-do-reino) — taxa configurável por kg, abatimento automático do pedido.
- Numeração de pedido por temporada (26-1, 26-2, ... reiniciando a cada safra).
- Auto-update do app Windows — fluxo completo de uma versão para outra com o servidor de updates.
- Importação da planilha histórica real — rodada em produção em 25/06; só a simulação com a cópia anonimizada tinha sido validada antes disso. O resultado real (quantos pedidos/pagamentos entraram, os 3 pagamentos de troca em pimenta que o script pula) não foi confirmado depois.

Recomendação: perguntar ao dono se tudo isso está funcionando bem em produção e, em caso positivo, atualizar as memórias correspondentes fechando os avisos de "não testado".

5. Ambiente de desenvolvimento local desatualizado

O arquivo `.env` local aponta para uma configuração de 08/06/2026 (localhost:3306, banco sistemapromudas) — bem anterior ao go-live de produção (25/06). Ao rodar a suíte de testes e2e do backend agora (npm test), todos os cenários falharam no login com erro 500 ("Erro interno do servidor"), indicando que o MySQL local não está no ar ou que o banco de desenvolvimento está fora de sincronia com o schema atual (migrations aplicadas em produção podem não ter sido replicadas localmente).

Na prática, isso significa que hoje não há como validar mudanças de backend localmente sem antes subir/ressincronizar o banco de desenvolvimento. Vale revisar isso antes da próxima mudança de código no backend.

6. Lacuna de acompanhamento (memória e uso real)

A memória do projeto e o histórico do git concordam entre si — ambos param por volta de 25-26/06/2026 — mas nenhum dos dois cobre as últimas ~6 semanas até hoje. Não é um caso de memória desatualizada em relação ao código; é uma lacuna real de acompanhamento do projeto nesse período.

- Última memória de estado do projeto: 25/06 (import-planilha-historica, auto-update-plano).
- Última confirmação de "banco de teste ou produção": 25/06, registrada como PRODUÇÃO — segue sendo a informação válida, mas nunca foi reconfirmada desde então.

7. Sugestão de plano para os próximos passos

- 1) Corrigir a rota /registrar pública (seção 2) — segurança, urgente, baixo risco, mas exige redeploy manual no servidor.
- 2) Confirmar com o dono o estado real de uso em produção nas últimas 6 semanas (bugs conhecidos? tudo funcionando?) e atualizar as memórias de acordo.
- 3) Sincronizar o ambiente de desenvolvimento local (`.env` / banco) para voltar a poder validar mudanças de backend antes de ir para produção.
- 4) Decidir o destino de `status_geral` e `recebedor` (dar uso real ou remover do schema).
- 5) Retomar (ou formalmente adiar) a Fase C do módulo Admin: permissões por usuário — verificar se ainda é prioridade do dono.
- 6) Avaliar se estoque de produtos virou uma necessidade real depois de ~6 semanas de uso em produção.
- 7) Revisitar a decisão sobre rodar offline (Relatorio-Estado-Projeto.pdf, Parte III) caso a rede da loja tenha causado algum problema nesse período.

Gerado automaticamente a partir da memória do projeto (MEMORY.md e arquivos individuais) e de uma inspeção do código/schema/histórico git atuais em 06/08/2026. Nenhuma alteração de código foi feita durante este levantamento.